Intel·ligència Artificial

Informe Pràctica 3: Planificació

Aleix Molins Teulé

Nathaniel Cruz Pascual

Maria Caldart Deu

Taula de continguts

1. Introducció....3
2. Representació del domini.... 4
2.1. Tipus d'objectes.... 4
1. Ciudad....4
2. Hotel....4
2.2. Predicats.... 4
1. en ?c....4
2. visitada ?c.... 4
3. viaje-finalizado....5
4. hotel-en ?h - hotel ?c - ciudad....5
5. alojado ?h - hotel....5
6. viaje-iniciado....5
7. vuelo ?c1 ?c2 - ciudad.... 5
2.3. Funcions (Fluents)....6
1. Acumuladors globals....6
2. Atributs.... 6
3. Control de l'estat local.... 6
3. Implementació dels operadors (accions).... 7
3.1. iniciar-viaje.... 7
3.2. extender-estancia....7
3.3. volar-y-alojarse....7
3.4. finalizar-viaje.... 8
4. Representació del problema.... 8
4.1. Declaració d'entitats....8
4.2. Estat Inicial....8
4.3. Objectiu.... 9
4.4. Funció de mètrica....9
5. Metodologia de Desenvolupament....9
6. Jocs de prova i Anàlisi.... 10
6.1. Joc de Prova 1: Itinerari d'estiu.... 10
6.2. Joc de Prova 2: Itinerari llarg....11
6.3. Experimentació amb les ponderacions (Extensió 4).... 11
7. Conclusions....12

1. Introducció

Aquest informe documenta el procés d'implementació d'un sistema de planificació automàtica de viatges per a l'Agència de viatges al fin del mundo i més enllà. L'objectiu principal d'aquest projecte és la síntesi i generació d'itineraris turístics entre diferents ciutats, de manera que siguin òptims. Donada una base de dades que conté ciutats, hotels i vols disponibles, el planificador dissenyat ha de traçar una ruta contínua que connecti diverses destinacions sense repetir-ne cap, assignant els allotjaments pertinents i respectant de forma estricta les restriccions logístiques i econòmiques imposades per l'usuari.

Aquesta tercera pràctica actua com a continuació de la pràctica anterior de Sistemes Basats en el Coneixement. Mentre que el SBC funcionava com un "diagnosticador" o filtre semàntic capaç de deduir el perfil de l'usuari (com per exemple una parella buscant un viatge romàntic o joves buscant diversió) i filtrar quines destinacions i hotels eren adequats a les seves preferències, aquest planificador actua com el "navegador" o optimitzador matemàtic. Un cop el SBC ha acotat l'espai de cerca als elements vàlids, el planificador resol la complexitat combinatòria de muntar la ruta més òptima per tal de garantir la viabilitat del viatge.

El desenvolupament s'ha dut a terme mitjançant una metodologia de prototipatge ràpid i disseny incremental, utilitzant el llenguatge de descripció PDDL i el planificador Metric-FF per poder operar amb variables numèriques tal i com es demana a l'enunciat. Hem partit de la implementació del nivell bàsic, afegint extensions fins a implementar els requisits de l'Extensió 4. Així podem assegurar que el nostre model respecta els límits de dies mínims i màxims per ciutat o pressupostos globals, i a més és capaç d'optimitzar de manera simultània l'interès turístic de la ruta i el preu total del viatge amb una funció mètrica ponderada.

En les seccions següents d'aquest document detallarem l'arquitectura del sistema. En primer lloc, explicarem com s'ha modelat el domini i la seva representació (tipus d'objectes, variables, predicats i accions), així com els problemes concrets. Finalment, exposarem l'anàlisi de diversos jocs de proves per tal de demostrar la robustesa del nostre model i la cobertura de les extensions.

2. Representació del domini

2.1. Tipus d’objectes

1. Ciudad

Representa cada node del graf sobre el qual es traça l'itinerari. Representa les destinacions físiques del viatge i és l'entitat principal sobre la qual s’imposen la majoria de les restriccions dures del sistema. Concretament, és on s'imposen les finestres temporals d'estada (mitjançant funcions de dies mínims i màxims per assolir l'Extensió 1) i on recau el pes de l'interès turístic per a l'optimització de la ruta (Extensió 2).

S'ha decidit modelar rutes unint instàncies individuals de ciutat en comptes de definir un macro-objecte "ruta/viatge" preestablert. Això és imprescindible per dotar el planificador d'una versatilitat total; el sistema té la capacitat de permutar i descobrir lliurement l'ordre de les ciutats, generant un viatge emergent adaptat a l'heurística d'optimització.

2. Hotel

Representa els diferents allotjaments vinculats a una destinació.

Separar lògicament l'allotjament de la pròpia ciutat és una exigència per tal de poder implementar correctament l'Extensió 3 (pressupost) i l'Extensió 4 (ponderacions). Si el preu fos un simple atribut intrínsec de la ciutat, l'usuari estaria forçat a assumir un cost fix per destí. En canvi, convertir l'hotel en un objecte autònom permet vincular múltiples allotjaments a una mateixa ciutat, cadascun amb un preu diferent. D'aquesta manera, permetem que el planificador pugui jugar amb la combinatòria microeconòmica: pot decidir allotjar-se en un hotel molt barat per estalviar pressupost i compensar el cost d'un vol més car o d'una ciutat d'alt interès.

2.2. Predicats

1. en ?c

Ens indica la ubicació actual de l'usuari, és a dir, la ciutat que estem visitant en aquell moment de la planificació.

És necessari per establir el node d'origen des del qual es buscaran les connexions de vols disponibles cap a la següent destinació, i per saber sobre quina ciutat s'han d'aplicar les sumes de dies d'estada.

2. visitada ?c

Actua com a registre històric per marcar que una ciutat ja ha estat trepitjada prèviament a la ruta.

És un mecanisme de control imprescindible per satisfer el requisit enunciat al nivell bàsic de "no repetir cap ciutat". S'utilitza com a precondició negativa (not (visitada ?dest)) a l'acció de volar, trencant qualsevol possibilitat de bucles infinits dins del graf de vols.

3. viaje-finalizado

Una de les condicions necessàries per tal que el viatge pugui acabar. S'inclou a Goal en el fitxer problem.pddl. La seva funció és garantir que l'última ciutat visitada també respecta la restricció del mínim de dies d'estada.

Com que la comprovació dels dies mínims es realitza regularment a l'acció de volar just abans de sortir d'una ciutat, l'última destinació, des d'on no s'arriba a sortir mai amb vol segons les assumpcions estipulades a l'enunciat, quedaria sense ser avaluada. Aquest predicat, que es genera exclusivament per l'acció finalizar-viaje, força el planificador a fer aquesta darrera validació temporal per poder concloure la ruta de forma lícita. El límit màxim de dies, per contra, s'assegura naturalment mitjançant la precondició de extender-estancia.

4. hotel-en ?h - hotel ?c - ciudad

Relaciona de forma estructural un allotjament concret amb la ciutat on es troba ubicat.

Aquest coneixement estàtic evita que el planificador pugui generar solucions incoherents, com per exemple estar ubicat a Barcelona però pagar i dormir a un hotel de París. Limita l'espai de cerca d'allotjaments només als vàlids per a la destinació actual.

5. alojado ?h - hotel

Emmagatzema l'hotel específic on l'usuari està passant les nits en l'etapa actual del viatge.

És el mecanisme que ens permet cobrar correctament el cost de l'habitació. Sense aquest predicat, l'acció d'estendre l'estada no sabria quin és l'hotel actiu ni quin preu (precio-hotel) ha de sumar a l'acumulador del pressupost total.

6. viaje-iniciado

Actua com un semàfor lògic de control de flux.

Ens garanteix que l'operador iniciar-viaje es pugui instanciar exclusivament una sola vegada durant el pas inicial (ja que té (not (viaje-iniciado)) com a precondició). Això evita que el planificador, de forma caòtica, intenti obrir rutes noves paral·leles o reinicialitzi variables a meitat de l'execució.

7. vuelo ?c1 ?c2 - ciudad

Defineix quines ciutats estan connectades per aire (les arestes del graf del problema).

És vital per limitar i delimitar els moviments permesos, assegurant que el planificador no s'inventi trajectes inexistents per fer drecera a l'hora de buscar la solució més ràpida o assequible.

2.3. Funcions (Fluents)

1. Acumuladors globals

Aquestes variables emmagatzemen l'estat evolutiu del viatge i són les que permeten verificar si es compleixen els objectius i aplicar la mètrica d'optimització.

- **total-precio**: Acumula els costos de tots els vols realitzats i de les nits d'hotel consumides. És una funció necessària per resoldre l'Extensió 3, ja que permet imposar les restriccions dures de pressupost mínim i màxim. També és un dels dos factors de la mètrica de l'Extensió 4.
- **total-dias**: Comptabilitza la durada total del viatge, sumant una unitat per cada nit d'estada. Resol el requisit de l'Extensió 1 que exigeix que el recorregut global tingui una durada mínima de dies, independentment de com es reparteixin entre les diferents ciutats.
- **total-interes**: Suma el valor d'interès de cada ciutat visitada. Permet al planificador avaluar matemàticament la "qualitat" de la ruta per poder minimitzar aquest valor global a la funció objectiu a l'Extensió 2.
- **num-ciudades-visitadas**: Comptador que s'incrementa cada cop que l'usuari aterra a una ciutat nova (i a l'inici del viatge). És el mecanisme de validació per complir el requisit del Nivell Bàsic, permetent exigir un nombre mínim de destinacions diferents a l'objectiu del problema.

2. Atributs

Aquestes funcions actuen com la base de dades on es consulta la informació de cada ciutat, hotel o trajecte. No canvien durant l'execució.

- **precio-vuelo**, **precio-hotel** i **interes-ciudad**: Restringeixen el viatge o ajuden a seleccionar entre diferents opcions. S'inicialitzen en el cas de prova.
- **min-dias-ciudad** i **max-dias-ciudad**: Defineixen l'interval en què es pot visitar una ciutat. Són necessaris per implementar l'Extensió 1.

3. Control de l'estat local

- **dias-estancia-actual**: Mira els dies que portem a la ciutat actual. A diferència del total de dies del viatge, aquesta es reinicia a 1 cada vegada que s'arriba a una nova ciutat. La seva existència és la que permet comparar en tot moment si s'està incomplint el mínim o superant el màxim d'estada de la ciutat actual abans de permetre un vol cap a la següent ciutat.

3. Implementació dels operadors (accions)

En aquesta secció llistarem els operadors que hem implementat i justificarem la seva necessitat alhora de resoldre correctament el problema demanat.

3.1. iniciar-viaje

Aquest operador s'encarrega de seleccionar la ciutat d'inici. No exigeix que es creï un vol anterior a aquesta ciutat (desde la ciutat origen de l'usuari) perquè s'especifica a l'enunciat que aquell vol no és part del problema.

- **Justificació de l'operador:** Aquest operador permet que el sistema de recomanacions no hagi de forçar a l'usuari a començar en un lloc concret, sinó que avalua quin inici és el més correcte segons la mètrica (per exemple, començar a la ciutat amb l'hotel més barat, si la mètrica diu que busquem ciutats barates).

L'operador suma 1 al comptador d'estada (:init l'inicialitza el comptador a 0 per tant el seu valor canvia a 1) i marca la ciutat de inici com a visitada per evitar la seva repetició a l'itinerari.

3.2. extender-estancia

Aquest operador permet al planificador decidir quedar-se un dia addicional a la ciutat actual.

- **Justificació de l'operador:** Permet que ens allotgem en una ciutat més de un dia (incrementant la funció dias-estancia-actual), sense aquest operador només podríem visitar cada ciutat un dia. La seva precondició és una desigualtat estricte (<) respecte al max-dias-ciudad, per garantir que no s'extenguin els dies més enllà del nombre de dies màxim. La flexibilitat d'aquest operador en estendre l'estada dia a dia és permet al sistema quadrar el total-dias al requerit al *goal*.

3.3. volar-y-alojarse

Aquest operador simbolitza el desplaçament entre ciutats i l'allotjament al nou hotel.

- **Justificació de l'operador:** L'acció de volar és necessària perquè és la única que ens permet anar d'una ciutat a l'altra (és a dir, incloure més d'un ciutat a l'itinerari). I l'acció de rebre allotjament també és sempre necessària perquè l'usuari necessita tenir un lloc per dormir a la ciutat.

  Agrupem el vol i l'assignació de l'allotjament en el mateix operador perquè l'arribada a la ciutat i l'assignació d'un allotjament són accions que no poden succeïr independentment (no existeix la opció d'estar a la ciutat sense hotel o d'estar a un hotel sense estar en una ciutat) per tant és lògic unificar les dues.

A més l'operador s'assegura de que la recomanació compleixi amb la restricció de dies mínims per ciutat. La precondició (>= (dias-estancia-actual) (min-dias-ciudad ?orig)) en la

transició de vol assegura que, abans de sortir de la ciutat, ens hi hem allotjat el nombre de dies mínim.

3.4. finalizar-viaje

Aquest operador és necessari per poder finalitzar el viatge (está inclòs a :goal) i s'assegura que l'última destinació compleix amb el nombre mínim de dies d'estada.

- **Justificació:** Com que les restriccions de dies mínims es comproven en l'operador volar-y-alojarse, l'última ciutat de l'itinerari queda sense supervisió (ja que no té un vol de sortida). Per això afegim aquesta mateixa precondició de (>= (dias-estancia-actual) (min-dias-ciudad ?orig)) a aquest operador, perquè d'aquesta manera no es pot acabar el viatge si no es visita l'última ciutat el nombre de dies mínim.

4. Representació del problema

En aquesta secció detallarem com s'ha estructurat la informació en els fitxers de problema (problem*.pddl), traduint els requeriments de l'usuari a l'estat inicial i definint l'objectiu a assolir en el bloc de :goal.

4.1. Declaració d'entitats

Es declaren les instàncies de ciudad i hotel. En els jocs de prova aportats es declara un hotel per ciutat, però es podrien afegir més.

4.2. Estat Inicial

L'estat inicial declara informació tant sobre les preferències de l'usuari, com la informació de hotels i vols (globals), com informació que s'actualitza dinàmicament durant el motor de recomanació (total-precio, total-interes, total-dias, ...).

- **Inicialització d'acumuladors:**

  Tots els fluents globals (total-precio, total-dias, total-interes, num-ciudades-visitadas, dias-estancia-actual) s'inicialitzen a 0. Això és vital perquè el planificador pugui realitzar operacions d'increment des del primer pas.

- **Configuració d'atributs:**

  Es defineixen els vols possibles oferts per l'agència de viatges usant el predicat (vuelo ?c1 ?c2) i se'ls hi assigna un cost amb (= (precio-vuelo ?c1 ?c2) X).

  S'estableixen les intervals d'estada (min-dias-ciudad i max-dias-ciudad) i l'interès turístic de cada destinació. Això permet seleccionar entre les ciutats a visitar segons els dies que interessen o l'interés.

4.3. Objectiu

L'objectiu representa les restriccions que el viatge ha de complir obligatòriament. Hem utilitzat comparacions numèriques per modelar els requisits de l'enunciat:

- **Restriccions de volum:** En Problem.pddl, (>= (num-ciudades-visitadas) 3) i (>= (total-dias) 6). Aquests valors forcen el planificador a generar un itinerari amb prou contingut, evitant solucions trivials de poca durada.
- **Restriccions del pressupost:** Rang de pressupost. En Problem.pddl, (<= (total-precio) 1500) i (>= (total-precio) 500).

  El límit superior garanteix que el viatge és assequible, mentre que l'inferior obliga al planificador a cercar rutes amb una qualitat mínima (hotels o vols millors).
- **Validació de finalització:** El requisit viaje-finalizado obliga el sistema a executar l'acció finalizar-viaje abans (ja que viaje-finalizado és un efecte d'aquest), garantint així que l'estada a l'última ciutat ha complert el seu mínim de dies.

4.4. Funció de mètrica

Per resoldre l'Extensió 4, incloem una secció (:metric ...) que fa que el sistema de recomanacions no només busqui un itinerari possible, sinó el millor dels itineraris possibles segons el criteri de la mètrica.

Hem introduit la mètrica ponderada:

$$(:metric\ minimize\ (+\ \ (*\ \ 0.1\ (total\ -\ precio))\ (*\ \ 10\ (total\ -\ interes))))$$

Justificació dels pesos: Com que el preu es mou en unitats de centenars (ex: 120€) i l'interès en unitats petites (ex: 1 o 2), hem aplicat multiplicadors per equilibrar la seva importància. Reduir el pes del preu (0.1) i augmentar el de l'interès (10) permet que ambdues variables influeixin de manera equitativa en la decisió heurística del planificador, evitant que una d'elles eclipsi l'altra.

5. Metodologia de Desenvolupament

Per abordar la complexitat de la pràctica, hem seguit una metodologia de disseny incremental. Aquest enfocament ens ha permès construir el domini capa per capa, assegurant-nos que cada funcionalitat era estable abans d'afegir la següent capa de dificultat.

El procés de treball s'ha dividit en les següents etapes:

1. **Fase inicial (Nivell Bàsic):** Vam començar definint l'estructura essencial: ciutats, hotels i vols. Vam aconseguir que el motor donés recomanacions de seqüències de ciutats (sense especificar els dies per ciutat).

   En aquesta fase teníem els operadors de iniciar_viaje y de volar-y-alojarse.

2. **Implementar l'Extensió 1:** Amb la base ja funcional, vam introduir el factor de dies per ciutat. Primer vam afegir l'operador d'estendre l'estada perquè construís sobre el nombre de dies de l'estada inicials. I després vam implementar la restricció de dies màxims a extender-estancia i la de dies mínims a volar-y-alojarse.

   Posteriorment, vam veure que per comprovar la restricció dels dies mínims en la última ciutat, havíem d'afegir la restricció viaje-finalizado al :goal. Degut a la manera que hem escollit de comprovar els dies mínims, la comprovació no es fa si no hi ha un viatge de sortida. Degut a això, vam haver d'afegir aquesta restricció.

3. **Implementar Extensions 2 i 3:** Un cop l'extensió 1 va ser robusta, vam incorporar els fluents de preu i d'interès. En aquesta fase, vam modificar :goal per imposar restriccions de pressupost (mínims i màxims) que filtressin els itineraris resultants. No vam implementar cap mètrica que minimitzés el preu o maximitzés l'interès, sinó que vam decidir implementar la mètrica ponderada de l'extensió 4 directament.

4. **Implementar Extensió 4:** Finalment, vam implementar la mètrica ponderada per resoldre l'extensió 4. Com ja hem explicat, vam ajustar els pesos de la fórmula per equilibrar el preu i l'interès, i en els experiments vam poder veure l'efecte que el canvi d'aquestes ponderacions té sobre el resultat final.

Durant tot el procés, vam utilitzar metric-ff per validar cada canvi. Aquesta validació contínua ens ha permès detectar errors ràpidament i garantir que el programa final fos robust.

6. Jocs de prova i Anàlisi

Per tal de validar el funcionament del nostre sistema i demostrar l'escalabilitat del domini, hem dissenyat dos problemes no trivials que cobreixen diferents casuístiques i restriccions.

Les traces d'execució completes d'aquests jocs de prova s'adjunten en el fitxer traces.txt lliurat amb la pràctica

6.1. Joc de Prova 1: Itinerari d'estiu

Aquest primer escenari representa un viatge curt on l'usuari vol visitar un mínim de 3 ciutats en un temps mínim de 6 dies totals, gastant entre 500 i 1500 unitats monetàries (problem.pddl).

El domini compta amb 4 ciutats (Barcelona, París, Londres i Roma), cadascuna amb els seus propis límits de dies (min-dias-ciudad i max-dias-ciudad).

El planificador resol el problema satisfactòriament en 7 passos (distància 0). El sistema escull començar a Barcelona, estén l'estada per complir els dies mínims, vola cap a Roma, i finalment vola a Londres.

El pla finalitza correctament executant l'acció finalizar-viaje a Londres, garantint que s'hi està el mínim de dies establert abans de tancar el viatge.

6.2. Joc de Prova 2: Itinerari llarg

Per provar l'escalabilitat del model, hem creat un segon escenari més complex (problem2.pddl).

Hem afegit una cinquena ciutat, Berlín, caracteritzada per tenir un allotjament molt econòmic (60/nit). Hem incrementat la dificultat demanant a l'usuari un mínim de 4 ciutats visitades i un mínim de 8 dies de viatge. També s'ha dissenyat un graf de connexions bidireccionals més ric, amb vols alternatius (ex: vols directes entre Barcelona i Roma) amb preus variats.

El planificador ha estat capaç de trobar solucions vàlides avaluant entre 23 i 31 estats, demostrant que el sistema és robust i no es queda encallat.

6.3. Experimentació amb les ponderacions (Extensió 4)

L'Extensió 4 ens demana optimitzar simultàniament el preu total i l'interès de les ciutats visitades.

Com que són unitats diferents, hem utilitzat ponderacions a l'expressió (:metric minimize ...). Hem executat el Joc de Prova 2 modificant els pesos d'aquesta mètrica per avaluar el comportament del planificador.

Experiment A - Prioritzar el preu: Mètrica*:* (+ (* 1 (total-precio)) (* 0 (total-interes)))

Anàlisi:

En donar tot el pes al preu, el planificador busca el camí més econòmic possible.Observem a la traça que escull començar a París i fer la ruta París -> Londres -> Berlín -> Roma, aprofitant al màxim la ciutat de Berlín, que té l'hotel més barat. El pla es completa en 9 passos i ignora per complet l'interès turístic.

Experiment B - Prioritzar l'interès: Mètrica: (+ (* 0 (total-precio)) (* 10 (total-interes)))

Anàlisi:

En anul·lar la importància dels diners, el planificador guia el viatge prioritzant les ciutats que tenen millor interès (valor 1).

La traça ens mostra una ruta totalment diferent: Barcelona -> Roma -> Berlín -> Londres. Com que l'únic que li importa és optimitzar l'interès, el pla resultant s'allarga fins a 11 passos (quedant-se molts dies extres a Londres) per quadrar l'equació de la millor experiència turística, ignorant que l'hotel de Londres és el més car de tots (180/nit).

Experiment C - Mètrica equilibrada: Mètrica: (+ (* 0.5 (total-precio)) (* 10 (total-interes)))

Anàlisi:

El planificador cerca un equilibri. En aquest cas concret, el cost desorbitat de l'hotel de Londres i els vols fan que el planificador torni a optar per la ruta de l'Experiment A (París -> Londres -> Berlín -> Roma), ja que penalitza massa el preu malgrat que l'interès d'algunes ciutats sigui millor.

Conclusió:

S'ha comprovat amb èxit que variar la ponderació de la mètrica dirigeix de manera efectiva la cerca heurística de Metric-FF, permetent obtenir plans fets a mida segons si es prioritza l'economia o la qualitat turística.

7. Conclusions

La realització d'aquesta pràctica ha permès consolidar els conceptes de planificació en l'espai d'estats mitjançant l'ús de llenguatges de representació formal com PDDL. Al llarg del desenvolupament, hem passat d'una implementació bàsica, a un sistema capaç d'optimitzar amb múltiples variables, i hem assolit amb èxit els objectius fins a l'Extensió 4.

El domini i els operadors, tal i com hem explicat en detall, són els necessaris perquè el sistema tingui el comportament de l'enunciat. I la seva implementació és robusta i té el seu comportament esperat en els experiments.

Finalment, en els experiments, hem vist que les restriccions del domini es reflecteixen correctament en el resultat. A més, les mètriques ponderades (que hem utilitzat com a criteri per seleccionar entre solucions) seleccionen correctament entre diferents tipus de solucions (ex: entre viatges on es seleccionen les ciutats amb més interès i els viatges més barats).